

An Autonomous ML Research Agent for KuaiRand-Pure

System Architecture

TikTok TechJam 2026, Track 2

Abstract

This document describes the architecture of an LLM-driven agent that runs an end-to-end machine learning research loop without human intervention: it inspects a dataset it has never seen, reproduces a published baseline, then repeatedly forms a hypothesis, writes the training script, executes it in a sandbox, verifies the result against a trusted evaluator, revises what it believes, and chooses what to try next. The design question it answers is not “can a model write a training script” but “what has to surround the model so that an unattended run produces a result you can trust.” Most of the system is that surrounding machinery: a trust boundary between the controller and model-written code, an evaluator that never believes a script’s own reported score, an integrity critic, a search tree with retirement, and a revisable belief set. Section 9 reports what the system has actually been measured to do — including one component that has never fired and one hypothesis the run records refute. Section 10 records where the implementation and the project’s own documentation currently disagree.

1 Overview

The task is within-user ranking of logged short-video impressions from the KuaiRand-Pure dataset. The scored label is the native column `long_view`; the metric is

$$\text{PRIMARY} = \frac{1}{2}(\text{GAUC} + \text{nDCG@5}),$$

with GAUC the positive-count-weighted mean of per-user AUC over users satisfying $0 < \text{positives} < \text{impressions}$, and nDCG@5 averaged over *all* users, so a user with no positive contributes zero. Splits are fixed by date: train 20220408–20220421 (1,141,112 rows), validation 20220422–20220428 (124,909), hidden test 20220429–20220508 (170,588).

The agent is dataset-agnostic in construction: everything it knows about KuaiRand is either measured from the cache at start-up (`agent/facts.py`) or established by its own experiments. The system is roughly 12,100 lines of Python with no third-party dependency in the controller itself — the LLM client is written against `urllib` — plus a 4,300-line React viewer that reads the run records off disk.

The training-data rule. One constraint runs through the whole design and is enforced at four separate layers, so it is worth stating once up front. Models may be fit on the **train split only** (20220408–20220421), for *both* the validation and the test predictions they save. Validation is feedback, not a second training set, and that extends beyond weights to anything *chosen* by watching validation — an early-stopping round count, a threshold, a blend weight. The four layers are: the proposer’s RULES block states it; the critic detects violations by AST (Section 5); `data_contract` in `run_meta.json` records which contract a run ran under; and the run viewer classifies runs under a superseded contract as **legacy-contract** rather than counting them.

1.1 Design constraints that shaped it

Four scored criteria drive most of the non-obvious decisions:

- **Autonomy** is measured by intervention count, so every failure mode has to be handled in-loop. A crashed script’s traceback is returned to the proposer with its own source; a timeout is fed back differently, because a timeout is not a bug to fix. Ideas are retired rather than escalated.

- **Cost** is scored, so the LLM is kept out of the training loop entirely. It is called twice per iteration in the sequential configuration — once to propose, once to revise beliefs — and everything else is deterministic Python. Prompt history is summarised to a fixed window so per-iteration token use stays flat as a run grows.
- **Feasibility** is scored on wall-clock, not accumulated script time, so the loop measures and enforces true elapsed time against a six-hour backstop.
- **Innovation** is scored on what *the agent* identified as worth trying, which is why the task brief is restricted to specification and API. Section 10 reports where that restriction currently leaks.

2 Repository layout

Module	Role	LOC	Trust
<i>agent/ — the product</i>			
loop.py	controller; the three phases, budgets, turn loop	955	trusted
proposer.py	prompt construction and response parsing	623	trusted
llm.py	stdlib LLM client, recording, replay	393	trusted
portfolio.py	slots, archive, refill, correlation gate	288	trusted
memory.py	distils prior run ledgers into context	229	trusted
knowledge.py	the revisable belief set	225	trusted
consultant.py	one cross-slot synthesis per turn	203	trusted
ensemble.py	incumbent retention and rank blending	196	trusted
tree.py	search over solution scripts	173	trusted
recovery.py	retry, then retire the idea	136	trusted
critic.py	integrity gate on a scored result	154	trusted
executor.py	sandboxed subprocess, credential stripping	110	boundary
selection.py	user-grouped fold split, accept/reject	108	trusted
diagnose.py	per-segment error profile, drift report	99	trusted
facts.py	measures dataset facts from the cache	86	trusted
kb.py	keyword retrieval over the paper catalogue	57	trusted
ledger.py	append-only JSONL, one record per iteration	52	trusted
<i>pipeline/ — the sandbox the agent works in</i>			
data.py	loader, fixed date splits, train-only vocabs	525	trusted
evaluate.py	GAUC / nDCG@5, pinned to the official metric	202	trusted
history.py	leakage-safe train-only entity histories	109	trusted
submit.py	submission writer and validator	141	trusted

Table 1: Principal modules. *Boundary* marks the single place where trusted control passes into model-written code. `report_run.py` (488) and the `viz/` React app read finished runs; `research/` holds human analysis explicitly off the submission path.

Reading a finished run. `viz/` is a local React application over the run records. Its landing view is derived entirely from `runs/`, never from the prose write-ups, so it cannot drift away from what the runs recorded. Its most useful piece is `run-index.ts`, which *classifies* every run folder from its own metadata rather than from a hand-maintained list: `submitted` (the run’s `submission.csv` is sha256-identical to `submission_best.csv`), `eligible`, `leakage` (`api_surface` exposed full-month item outcome columns), `legacy-contract`, `bonus-dataset`, `unverified` (no `run_meta.json`, so the contract cannot be checked), and `scratch`. `unverified` is deliberately not a verdict — those runs are neither claimed nor excluded, because the records alone cannot settle it.

3 The control loop

`run_agent.py` builds the collaborators — proposer, evaluator, ledger, tree, recovery, knowledge — and hands them to `agent.loop.run_loop`, which owns the entire run. Figure 1 shows one iteration end to end.

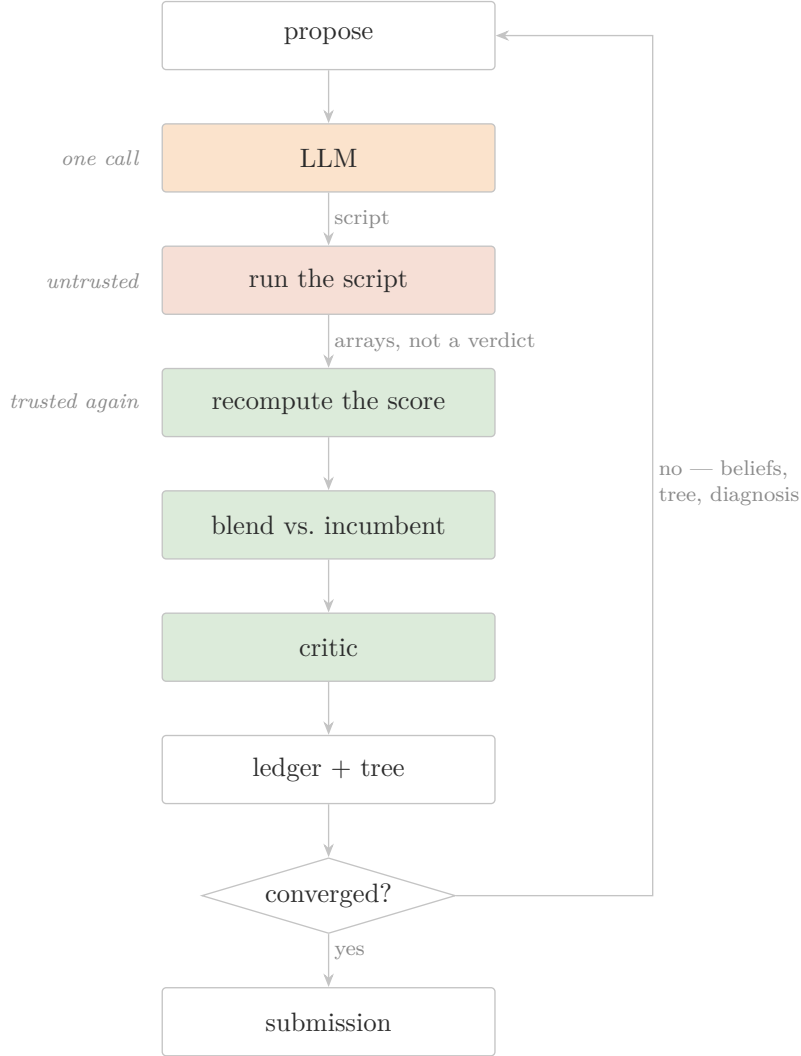


Figure 1: One iteration. The arrow leaving the sandbox carries *arrays, never a verdict*: a generated script is part of the search space, not part of the evaluator. Nothing reaches the ledger, the tree or the submission until the score has been recomputed, blended by the controller and passed by the critic.

3.1 Phases

`run_loop` walks three phases in order, choosing on each pass of the `while` loop (`loop.py:534–545`):

1. **eda** — the agent writes and runs its own inspection script. Judged only on whether it ran and printed something; there is no score. Its stdout is persisted to `eda_report.txt` and injected into every later prompt. This is the agent’s *only* source of dataset knowledge. Two attempts.
2. **baseline** — the agent stands up an end-to-end Factorization Machine pipeline. Accepted only if it lands within 0.002 of the organizers’ published validation PRIMARY = 0.6016. A score far *above* the baseline is fed back as a suspected leak, a score far below as a pipeline error. On success the script becomes the root of the search tree. Four attempts, after which the run stops with `baseline_not_reproduced`.

3. **improve** — the turn loop of Figure 1, until a stop condition fires.

A run terminates on the first of: convergence, the 50-script cap, the six-hour wall-clock backstop, an exhausted proposer, a daily LLM quota, or a broken environment (five consecutive sub-second failures with no output at all).

3.2 The convergence rule

The organizers specify that a run has converged when the validation score “has not improved by more than $\varepsilon = 0.002$ over the last $N = 3$ consecutive iterations.” This is read as written — the gain *across* the window, not each single step clearing ε (`loop.py:479`):

$$\text{converged} \iff |c| > N \wedge c_{-1} - c_{-1-N} \leq \varepsilon,$$

where c is the running best-validation curve. The distinction is material: a run climbing $+0.0008$ per iteration has improved by 0.0024 over three and is not converged, whereas the per-step reading would stop it. The stricter reading is nonetheless recorded as **strict_convergence_iteration** in `run_meta.json`, so a run can be checked against either. Both ε and N are also written to the metadata, so a diagnostic run launched with `--patience 999` is distinguishable from a default run that simply never converged.

4 Prompt construction

`proposer.py` assembles the prompt from ordered blocks. The ordering is not cosmetic: providers discount input that repeats as a stable *prefix*, so blocks that are identical across a run — the task brief, the paper catalogue, the parent script — are placed first, and blocks rewritten every iteration follow. The same text is sent either way; only the cache hit rate differs.

Block	Contents
TASK_BRIEF	Metric definitions, split row counts and date windows, the pipeline API signature by signature, the output contract, the training-data rule, the environment. Numbers are interpolated from <code>agent/facts.py</code> , measured from the cache, so pointing the harness at another KuaiRand variant does not feed it false premises.
phase instruction	<code>_EDA_TASK</code> , <code>_BASELINE_TASK</code> or <code>_IMPROVE_TASK</code> .
literature catalogue	All 33 entries, one line each — “it lists every method available, including ones that may not suit this data; deciding which are relevant is your job.”
EDA output	The agent’s own inspection results, truncated to 4000 characters.
parent script	The chosen node’s <i>actual source</i> , in full (14 000 characters). This is what makes iteration $N+1$ able to build on N rather than restarting from a skeleton.
diagnosis	<code>diagnose.segment_report</code> : the incumbent’s nDCG@5 against the per-user ceiling, bucketed by impression count, with each bucket’s share of the total gap.
history, beliefs, blacklist	The last 8 iterations relabelled by what happened to the <i>score</i> rather than the loop status; the current belief set; retired ideas.
mode instruction	<code>_SWEEP_</code> , <code>_TUNE_</code> , <code>_BROADEN_INSTRUCTION</code> .
siblings, seed note	Portfolio only: what other slots are attempting this turn; why a revived slot is resuming.

Table 2: Prompt blocks, in assembly order.

The response contract is one **HYPOTHESIS:** line and exactly one **python** fence. A reply that fails to parse, or whose hypothesis matches a retired idea, is retried once with a corrective note; a second failure returns **None** and the run stops with **exhausted**.

5 The trust boundary

The generated script is treated as adversarial in three distinct ways, because the three risks are independent.

Credentials. Model-written code runs unsandboxed with the controller’s environment. A measured run exposed `OPENAI_API_KEYS`, `GITHUB_PACKAGE_TOKEN` and others to the child. `executor._strip_credentials` therefore denies by pattern (`KEY`, `TOKEN`, `SECRET`, `AUTH`, ...) rather than allowing by name: an allowlist of what a training script needs is impossible to keep correct across platforms, and getting the denylist slightly wide only costs a variable nothing needed.

The hidden test set. `AGENT_HIDE_TEST_LABELS=1` makes `load("test").y` return a `HiddenTestLabels` stand-in that keeps `len` and `shape` — legitimate scoring code allocates against them — while every conversion, index or attribute access raises a `RuntimeError` naming the rule. `load("test").aux` is similarly name-visible and value-hidden. Selection on the hidden split is thus not a policy the model is asked to respect but a capability it does not have.

The reported score. `SavedScoresEvaluator` (`loop.py:104–190`) requires the script to save the exact arrays behind its metrics, reloads them, checks shape, finiteness and length against the split, and recomputes `PRIMARY`, `GAUC` and `nDCG@5` with the pinned evaluator. The script’s own `METRICS` line is compared to that at a 10^{-6} tolerance — tight enough that a fabrication worth making is caught, loose enough that a script printing six significant figures is not. The tolerance was 10^{-9} and cost a legitimate iteration to a 5×10^{-7} display-rounding gap.

Validation reaching a fitted quantity. The critic’s fourth rung is the subtlest and was added last. An audit of 36 submitted scripts found two that fit weights on train but chose the *round count* by early stopping against `valid_sets=[<validation>]` — and then scored and reported those same validation rows. The critic now walks every call node for a `valid_sets`, `eval_set` or `validation_data` keyword whose value references a variable bound to `load("valid")`, and rejects it. What makes this rung necessary rather than redundant is that the effect is small: measured at +0.00041, roughly half a seed sigma, so it never trips the score-jump rung and nothing else in the chain was looking for it. The feedback returned to the proposer names the remedy — hold out the last days of *train* instead.

An externally checkable claim. `research/leakage_check.py` (28 lines) exists so that a reader does not have to take the preceding four layers on trust. It fits one `LightGBM` configuration twice — train-only and train+validation — and prints both splits’ scores. If the agent were training on validation, its reported validation number would sit near the train+valid row rather than the train-only row. The submitted run’s 0.605309 is printed alongside for comparison. The design intent is stated in the module docstring: “*Run this yourself; trust the output, not us.*”

Leakage in the features themselves is handled a layer down. The post-click signals (`play_time_ms`, `is_click`, `is_like`, ...) are outcomes of the row being scored; `pipeline/data.py` exposes them only as `Split.aux`, never in `Split.X`, and the dataset’s supplied `video_features_statistic` file is not loaded at all because its counts aggregate over the full month and overlap the evaluation windows. `pipeline/history.py` recovers the same mechanism safely: entity histories aggregated over train rows only, leave-one-out on train so an example never receives its own target.

6 Search over solution scripts

Each scored script is a node in `tree.py` carrying its source, hypothesis, score, and counts of children, misses and crashes.

- **Miss.** A child failing to clear its parent by ε increments `misses`; two retire the node.
- **Crash.** A child that crashed or timed out counts half — the idea may be sound and the code wrong, and recovery already retries it — so four retire the node. Without this, a node that produced only crashes never approached retirement; one historical node spent five children and 78 minutes that way, one of them timing out at 4,373 s.
- **Selection.** `select("refine")` returns the highest-scoring live node; `select("sweep")` returns `min(live, key=(children, -score))` — the least-explored node, ties broken toward the higher score. Retirement plus this fallback is the backtracking a linear greedy walk cannot do.

`max_misses` defaults to 2 and must stay strictly below `patience`. When both were 3, checked against the same ε against the same reigning leader, a node was retired on exactly the iteration the run converged, and `select` never got to return the fallback — backtracking existed only for crash-heavy nodes.

Idea retirement. `recovery.py` keys an idea by the *method it names*, not the sentence, via an alias table (`lambdamart`, `ranknet`, `listwise` → `lambdarank`) and an overlap-coefficient match at 0.6. Overlap rather than Jaccard, because the model restates one idea at very different lengths and Jaccard reads a wordier rephrasing as a new idea. Two retries; six if the family has already scored well, on the reasoning that it is failing on code bugs rather than on being wrong. An idea that runs cleanly but loses to the incumbent by more than 2ε three times is retired too — crash-based blacklisting never fires for code that works.

7 Belief revision

After each scored iteration the agent rewrites a bounded set of claims, each with a status of `active`, `qualified` or `invalidated` and a list of supporting iteration numbers (`knowledge.py`). The point of the status field is that later evidence can *demote* an earlier conclusion rather than accumulating beside it; an append-only reflection log produced runs that recommended the same next experiment four times because nothing could overturn it. The rendered set — not a human-authored brief — is what guides the next proposal.

Two properties are enforced by construction: the set is capped at 14 claims so token use stays flat, and revision never raises. An LLM outage, a non-JSON reply or a malformed element leaves the previous belief set standing rather than ending an unattended run.

8 Portfolio mode

With `--slots n` for $n \in \{2, \dots, 5\}$ the loop advances n lineages per turn. Figure 2 shows the turn structure.

The justification for the whole portfolio is falsifiable and is measured every turn. `pairwise_rank_correlation` computes within-user rank correlation between the slots' pre-blend validation predictions; ranks are taken inside each user because the metric only ever compares rows belonging to the same user. If that number sits above 0.95, the slots are ranking validation identically and the extra cost bought nothing. Measuring it *pre-blend* matters: one run read 1.000 because two slots had blended to $\alpha = 0$ and were literally holding the same incumbent array.

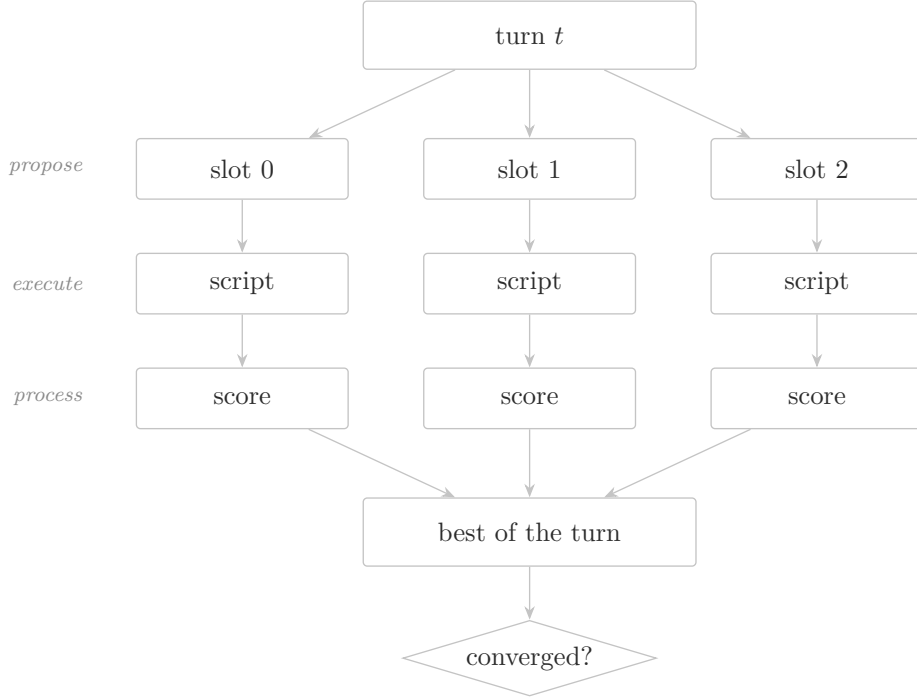


Figure 2: One portfolio turn. Proposals are drawn one slot at a time (the LLM client mutates shared key-failover state), the scripts then run *concurrently*, and their results are processed back in slot order so a later slot can blend against an earlier one’s result. The organizers’ stopping rule is singular — there is no per-agent counter in it — so n lineages cannot each carry one. What they *can* do is share a turn: the turn’s score is the best of its slots, and that single curve is what the rule is measured against. Stale slots are archived and refilled between turns.

9 Measured behaviour

9.1 The submitted run

	GAUC	nDCG@5	primary
Official baseline, hidden test	0.6610	0.5282	0.59460
Run r79 iteration 4, hidden test	0.6673	0.5321	0.59969
Delta	+0.0063	+0.0039	+0.00509

Table 3: Selection was on validation only (0.60531 against the baseline’s 0.6016); the hidden test set never chose the winner. Cost: 6 iterations of the 50 allowed, 20.2 minutes wall-clock, 96,359 tokens, CPU only, 0 failures, 0 manual interventions. Verified against `runs/r79/run_meta.json`.

9.2 The slot ladder

9.3 Where the gain actually comes from

The controller-side blend does most of the work, and not where the design expected. In **r87**, iterations 6 and 7 both produced candidates scoring *below* the standing incumbent and still advanced the leader, because `retain_or_blend` found a mixture that beat both:

Run	Slots	Scripts	Wall-clock	Validation	Hidden test
r85	1	8	10.8 min	0.604424	0.598788
r86	2	11	15.4 min	0.604452	0.599473
r87	3	14	22.0 min	0.605713	0.600410
r88	4	18	27.4 min	0.605090	0.600194
r89	5	22	32.9 min	0.605716	0.600256

Table 4: All four turns each, so the comparison is at equal turn budget. Re-derivable with `python -m research.verify_claims`, which re-checks these against the run records and reports disagreements.

Iteration	Candidate	Incumbent	Selected α	Retained
6	0.6045	0.6043	0.50	0.6049
7	0.6046	0.6049	0.25	0.6052
9	0.6055	0.6052	1.00	0.6055
10	0.6053	0.6055	0.00	0.6055

By contrast the *portfolio-level* blend of Figure 2 step 5b has never once been accepted: 0 of 40 attempts across all ten portfolio runs, every one recording "reason": "no member improved fold A", and `portfolio_blend_primary` null in every `run_meta.json`. The fold-A/fold-B gate is doing exactly what it was written to do — it is refusing blends that do not clear ε on the selection fold — but the consequence is that this component contributes nothing at present and should either be re-tuned or removed.

9.4 A refuted hypothesis: decorrelation does not predict the score

Section 5 argued that the portfolio is worth its cost only if its slots disagree, and the correlation gate was built as the acceptance test for that argument. The gate was worth fixing — measured post-blend it was reading 1.000 by construction — but it does not follow that the number it now produces is worth *optimising*, and the run records refute the stronger claim.

Run	Configuration	Mean corr.	Validation	Hidden-test delta
r90	gate broken; slots were near-clones	~0.99	0.605282	+0.00530
r87	gate fixed	0.741	0.605713	+0.00581
r92	gate fixed + drift axis	0.276	0.605454	+0.00469

Table 5: Three converged 3-slot runs. r92 produced the best decorrelation on record — per-turn means 0.7794, 0.6687, 0.6991, 0.2757, under the 0.90 “proceed” line on every turn — and the *worst* hidden-test delta of the three. All figures re-derived from `portfolio.jsonl` and `run_meta.json`.

Both directions are now measured. Slots that were literally copies of one another scored +0.00530; slots that disagreed as much as anything the harness has produced scored +0.00469. Diversity is not the axis the score moves along, and a run cannot be steered by pushing the correlation down. The remaining spread across r79, r90 and r92 is 0.0006 with no trend, which is seed noise. Cost of establishing this: three converged runs, roughly 90 minutes and 830k tokens. What it buys is the end of a line of investigation — recorded in `docs/BUGS.md` rather than quietly dropped, because a negative result that closes a direction is worth as much as the positive one that opened it.

9.5 Cross-run memory is off by default

`memory.py` distills prior run ledgers into the next run’s context. It is now disabled unless `--memory` is passed, and the loop prints `cross-run memory: OFF` rather than staying silent, since a silent absence reads as “there was no history to find.” The reasoning is a rules argument rather than

a performance one: the benchmark budgets 50 iterations *per run*, and a run inheriting several hundred scored experiments from earlier ones is not the N -iteration run its own metadata reports. Which folders sit in `runs/` is also a human decision, so memory routes human judgement into a run through a channel `manual_interventions` cannot see.

10 Documentation and implementation divergences

The following were found by reading the code against `README.md` and confirmed against the run records. They are recorded here rather than silently corrected because several of them bear on claims the project makes about itself.

1. The “no human priors” guarantee does not cover the live prompt. `README.md` states that “nothing in the agent’s prompt tells it what works on this dataset,” guarded by `test_brief_carries_no_human_findings`. That test scans `TASK_BRIEF` only (`tests/test_proposer.py:38`). Every improve iteration also appends `_SWEEP_INSTRUCTION` (`proposer.py:285`, injected at :581), which contains hand-written measured findings — “*uniform days score 0.4597 and a 4-day half-life scores 0.5518*”, “*Seventeen architectures have been tried*”, “*selection protocol REFUTED, do not spend an iteration on it*”, “*the blend has beaten every standalone family, every time*” — and names specific architectures. Running the test’s own banned-term list against the blocks it does not scan:

Block	Banned terms present
<code>_SWEEP_INSTRUCTION</code>	blend, lambdarank, deepfm
<code>_TUNE_INSTRUCTION</code>	blend
<code>_BROADEN_INSTRUCTION</code>	—

It does reach the model: 12 of `r87`’s 12 improve prompts contain the block. Either the guarantee should be narrowed to “the task specification carries no findings,” or the guard should be extended over every block `_build_prompt` can emit — which would require rewriting the sweep instruction.

2. Three of the four search modes are unreachable in a normal run. `loop.py:571` sets `mode = "sweep"` unconditionally for every improve iteration. `_EDIT_PARENT`, `_BROADEN_INSTRUCTION` and `_TUNE_INSTRUCTION` fire only under the diagnostic `--force-mode` flag. Prompt-block counts over `r87`: sweep 12, broaden 0, tune 0, edit 0. The `README` loop diagram still presents “adaptive: refine or broaden” as the live policy, and the FML-bench citation supporting adaptivity describes behaviour the current configuration does not exhibit. The literature justification for the sweep-only policy is sound and is stated in the code comment; the diagram is simply stale.

3. No run exists yet under the current data contract. This gap has narrowed but not closed. The `train-only-v3` contract is now committed and enforced on three sides: `run_agent.py:47` sets it, the proposer’s `RULES` block was rewritten from “the TEST scores you save may come from the SAME recipe refit on train + validation” to “TRAINING DATA IS THE train SPLIT ONLY,” and the critic gained the `eval_set` rung to enforce it. What remains is that *every* run in `runs/` — including `r79`, the source of `submission_best.csv` — still records `train-plus-valid-v2`, the contract the code’s own comment calls non-compliant with benchmark rule 2.9.2. The viewer classifies these honestly as `legacy-contract`, so nothing is being concealed, but the submission has to be regenerated under `v3` before the compliance claim holds end to end. This is the single highest-value outstanding item.

4. The convergence sweep is set up but not run. The `.gitignore` merge conflict noted in an earlier revision of this document has been resolved. The analysis code, `research/convergence_sweep.py` (256 lines), is finished and well-constructed — it replays

every (ε, N) cell offline against an uncensored `--patience 999` curve so no grid cell costs a run, and it imports `agent.loop.converged` rather than keeping a second copy of the rule. But the three `runs/convergence_sweep/curve_*` inputs are committed while still incomplete: 3–4 ledger rows each against an intended 50-script curve, no `run_meta.json`, zero-byte logs. The study cannot be run until those three curves are regenerated to the cap.

5. Minor. `consultant.py` describes a “28-paper catalogue”; `kb/papers.json` holds 33. `README.md` still documents `--no-memory` as the way to disable cross-run memory, which now parses but is the default; and its loop diagram is the source of divergence 2 above. The `README` team table is still a `TODO(team)` placeholder, and it is a required deliverable.

Verified as accurate. All 23 test modules pass after the contract and critic changes. `r79`’s headline numbers match its metadata exactly. Every figure in the `docs/BUGS.md` decorrelation table reproduces from `portfolio.jsonl` and `run_meta.json` to the digit. `research.verify_claims` checks 34 claims and reports one problem (`r30` unverifiable for lack of a submission block — an absence, not a disagreement). The cost profile is as described: 11 LLM calls for 6 scripts at one slot, and n proposals plus one consultant call per turn in portfolio mode.

11 Reproducing a run

```
# data (194 MB, public)
mkdir -p data/raw && cd data/raw
curl -L0 https://zenodo.org/records/10439422/files/KuaiRand-Pure.tar.gz
tar -xzf KuaiRand-Pure.tar.gz && cd ../../
python -c "import pipeline.data as d; \
          d.build_cache('data/raw/KuaiRand-Pure/data', 'data/cache')"

export OPENAI_API_KEY=...          # and LLM_PROVIDER=openai, or ANTHROPIC_API_KEY

python -m pipeline.baseline_fm      # the official baseline, for reference
python run_agent.py --run-dir runs/rN --iters 50 --timeout 900 --patience 3
python report_run.py runs/rN > RUN_REPORT.md
python -m pipeline.submit --check --in runs/rN/submission.csv --split test

python -m research.leakage_check    # train-only vs train+valid, side by side
python -m research.verify_claims    # re-derive the ablation tables
```

Cross-run memory is off by default; pass `--memory` to opt in. `--slots n` selects portfolio mode. `--force-mode` pins the search mode for a diagnostic run and is not for scoring.

Two offline modes exist. `--dry-run` exercises the entire loop with a canned LLM and no network. `--replay runs/rN/llm_calls.jsonl` re-runs a previous run against its recorded responses: deterministic, no tokens, and it reproduces the original ledger to the last decimal. Replay tests the loop, the parsers, the ledger and the reporting — *not* prompts, since a changed prompt still receives the response recorded for the old one. `--replay-strict` fails the moment a prompt diverges, so that limitation cannot pass unnoticed.